

LAB#06**INFORMED SEARCH TECHNIQUES**

Implement informed A* Search, Best First Search and Greedy Best First Search algorithms using Python.

I. Informed A* Search

A* search is an informed search algorithm used for path-finding and graph traversal. It combines the advantages of both Dijkstra's algorithm (in that it can find a shortest path) and Greedy Best-First-Search (in that it can use a heuristic to guide search). It combines the information that Dijkstra's algorithm uses (favoring vertices that are close to the starting point) and information that Best-First-Search uses (favoring vertices that are closer to the goal).

Let $g(n)$ represent the exact cost of the path from the starting point to any vertex n , and $h(n)$ represent the heuristic estimated cost from vertex n to the goal. Dijkstra's algorithms build a priority queue of nodes ordered on their $g(n)$ values. Best-First-Search employs a priority queue of nodes ordered on $h(n)$ values.

A* balances the two as it moves from the starting point to the goal. It builds a priority queue of nodes ordered on $f(n) = g(n) + h(n)$ which is the total estimated path cost through the node n .

a) ALGORITHM: Informed A* Search:

Input: $start, goal(n), h(n), expand(n)$
Output: path

```

1 if  $goal(start) = true$  then return  $makePath(start)$ 
2
3  $open \leftarrow start$ 
4  $closed \leftarrow \emptyset$ 
5 while  $open \neq \emptyset$  do
6    $sort(open)$ 
7    $n \leftarrow open.pop()$ 
8    $kids \leftarrow expand(n)$ 
9   forall the  $kid \in kids$  do
10     $kid.f \leftarrow (n.g + 1) + h(kid)$ 
11    if  $goal(kid) = true$  then return  $makePath(kid)$ 
12    if  $kid \cap closed = \emptyset$  then  $open \leftarrow kid$ 
13   $closed \leftarrow n$ 
14 return  $\emptyset$ 
```

b) Algorithm Explanation:

- First place the starting node into OPEN and find its $f(n)$ value.
- Then remove the node from OPEN, having the smallest $f(n)$ value. If it is a goal node, then stop and return to success.
- Else remove the node from OPEN, and find all its successors.
- Find the $f(n)$ value of all the successors, place them in OPEN, and place the removed node into CLOSE.
- Repeat Step-2.
- Exit.

c) OPEN nodes:

An array that contains the nodes that have been generated but have not been yet examined.

d) CLOSE nodes:

An array which contains the nodes which are examined.

II. Greedy Best First Search

The greedy best-first search algorithm is incomplete without cycle checking. If a closed list is used to not visit nodes which have already been visited, then the search is complete. The algorithm is also not optimal. Since the algorithm is only based on the heuristic value, if the heuristic is not exactly equal to the actual cost, it might go down a less optimal path than the ideal one.

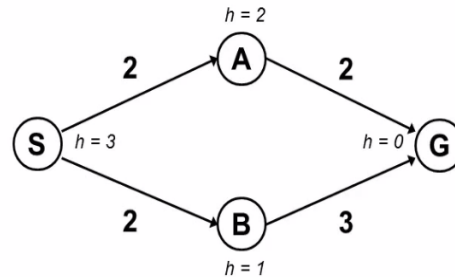
The greedy best-first search algorithm is $O(bm)$ in terms of space and time complexity. (Where b is the average branching factor (the average number of successors from a state), and m is the maximum depth of the search tree.) A good heuristic can bring this down substantially for the common case, but all nodes are visited in the worst case.

III. Best First Search

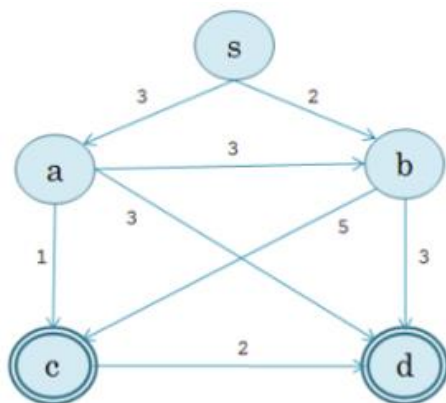
The best first search uses the concept of a priority queue and heuristic search. It is a search algorithm that works on a specific rule. The aim is to reach the goal from the initial state via the shortest path. The best First Search algorithm in artificial intelligence is used for finding the shortest path from a given starting node to a goal node in a graph. The algorithm works by expanding the nodes of the graph in order of increasing the distance from the starting node until the goal node is reached.

Activity- 1:

Implement A* search in the following state graph, find the shortest path between node S to Node G. Also solve it manually.

**Activity- 2:**

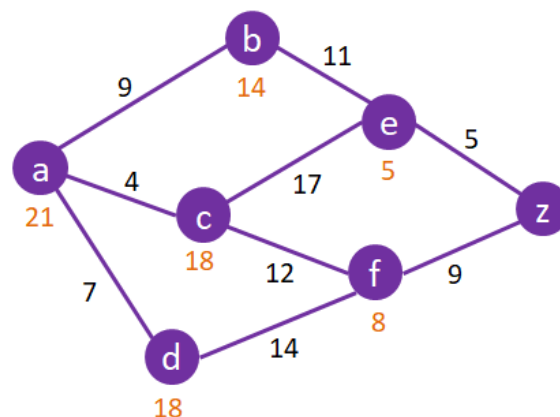
Implement A* search in the following state graph, find the shortest path between node 's' to node 'c' or 'd'. Also solve it manually.



h(s)	h(a)	h(b)	h(c)	h(d)
1	3	3	0	0

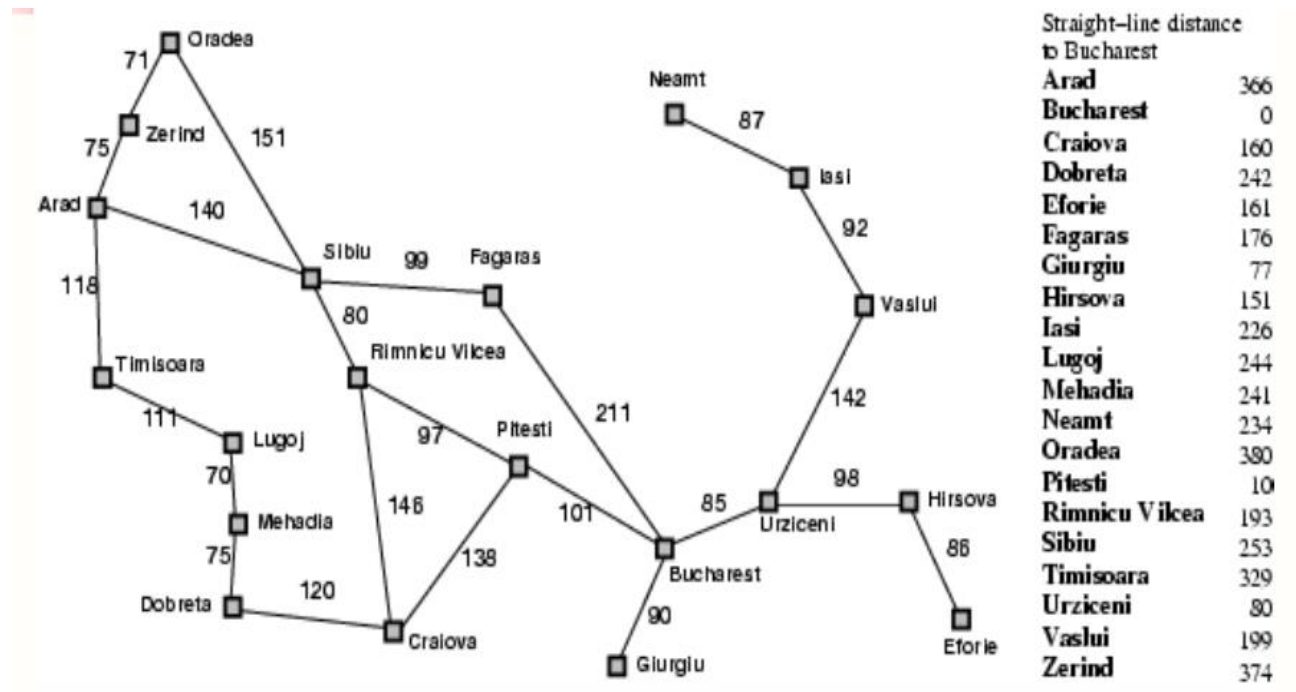
Activity- 3:

Implement A* search in the following state graph, find the shortest path between node 'a' to node 'z'. Also solve it manually.

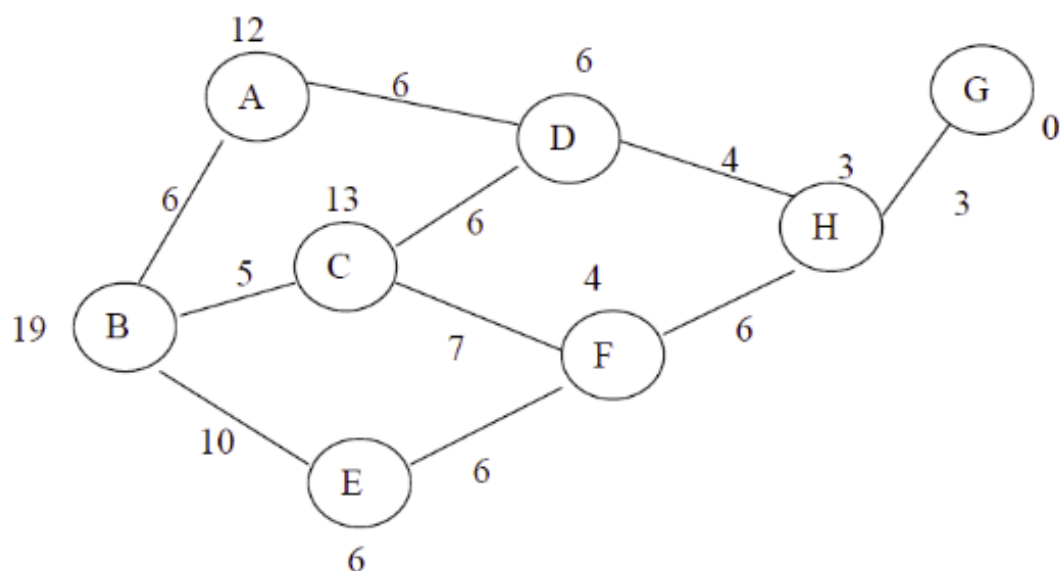


Activity- 4:

Implement A* search in the following state graph, find the shortest path between node Arad to Node Bucharest. Also solve it manually.

**Activity- 5:**

Implement Greedy Best First Search in the following state graph, find the shortest path between node 'A' to Node 'G'. Also solve it manually.



Activity- 6:

Implement Best First Search in the following state graph, find the shortest path between node 'S' to Node 'G'. Also solve it manually.

